

Purpose:

• Measure PIC16F883 instruction timing, use instruction-cycle calculations to create precise software delays, extend those delays with nested loops, and then refactor the working delay into reusable subroutines.

• The lab progresses from measurement to prediction, implementation, verification, and reusable program structure.

Standards and references:

- RCET 3375 Lab Standard
- RCET PIC-AS Style Guide
- PIC 16F883 Data sheet
- PICmicro Mid-Range MCU Family Reference Manual
- DLG 7137 datasheet

} GitHub
links

Equipment & materials

- MPLAB X IDE and pic-as toolchain
- PIC kit programmer/debugger
- PIC16F883 circuit from Lab 2
- 4 MHz crystal^{LG} oscillator circuit
- DLG 7137 display
- Oscilloscope
- Frequency counter
- Digital multimeter
- Lab book
- Optional - Logic analyzer

Part 1 - Measure Instruction TimingGoal:

• Use port B counter program from Lab 2 to measure the execution time of PIC16F883 instructions.

Use the RB0 output waveform to:

- Measure pulse width (PW), pulse space (PS), period, and frequency.^{LG}
- relate the measured waveform to the instructions executing in the program loop.
- determine the actual instruction-cycle time, T_{cy} .
- calculate the actual oscillator frequency.
- verify the effect of adding and removing NOP instructions.

The purpose of this Lab^{LG} part is to connect assemble instructions to real, measurable time.

LG

Before Lab:

- Start with the working PORTB counter program
- Review the instructions executed repeatedly in the main program loop.
- For each instruction in the repeating part
 - 1) identify how many instruction cycles it requires.
 - 2) determine how many instruction cycles occur between changes of RBO.

• The PIC instruction clock is related to the oscillator by

$$\text{Time}_{\text{period}} = T_{\text{osc}} = \frac{1}{F_{\text{osc}}} \quad \text{and} \quad T_{\text{cy}} = 4 \cdot T_{\text{osc}}$$

Frequency

$$\rightarrow F_{\text{osc}} = \frac{4}{T_{\text{cy}}} \quad 4 \cdot (4\text{MHz})^{-1} = 1\mu\text{s} = T_{\text{cy}}$$

4 or 250ns

• The circuit uses a nominal 4MHz crystal. Calculate the expected instruction-cycle time before making measurements.

- Determine the ^{LG} ~~expected~~ expected number of instruction cycles for
 - RBO PW
 - RBO PS
 - One complete RBO period.

• Predictions -

$$1\mu\text{s} \cdot 5\text{PCY} = 5\mu\text{s per instruction cycles}$$

$$5\mu\text{s} = \text{PW \& PS}$$

$$5\mu\text{s} + 5\mu\text{s} = 10\mu\text{s} = \text{period}$$

In the Lab:

1) Measure the Lab 02 counter

• Measure and record (^{LG} RBO/LSB) of counter output:

- PW
- PS
- period
- frequency

• Capture an oscilloscope image showing enough of the waveform to verify the ^{LG} measurements.

• Compare PW & PS and explain why RBO changes state at the observed rate based on the binary count and the instruction in the main loop.

2) Determine the instruction cycle time

• Use the measured waveform and the instruction-cycle analysis of the main loop to calculate the actual instruction cycle time.

• If N instruction cycles occur during one measured interval

$$\rightarrow T_{\text{cy}} = \frac{t_{\text{measured}}}{N}$$

• If you used the complete waveform period, account for all instruction cycles executed during the entire period.

• Don't assume $T_{cy} = 1\mu s$ for this measurement. Determine T_{cy} from the actual waveform.

3) Determine the oscillator frequency

• Calculate the actual oscillator frequency from the measured instruction-cycle time:

$$\rightarrow F_{osc} = \frac{4}{T_{cy}}$$

• Compare the result with the nominal 4MHz crystal frequency.

• Calculate the % difference:

$$\frac{(F_{measured} - F_{nominal})}{F_{nominal}} \times 100$$

• Explain possible reasons why the measured oscillator frequency is not exactly 4.000MHz

$$\frac{(3.99995MHz - 4MHz)}{4MHz} \cdot 100 = 0.00125\%$$

• The capacitors c_1 and c_2 could have small differences and if these values aren't @ $15pF$ exactly. There can also be stray capacitance.

4) ~~4~~⁴ Add & remove NOP

• Add one NOP instruction to the repeating execution path of the main loop. Before programming the PIC, predict the resulting change in PW, PS, period, and frequency. Build and program the modified code, then measure RBO again. Compare the measured timing change with the instruction cycle time determined earlier. Remove the NOP, program the PIC again, and verify that the waveform returns to its previous timing.

• Adding 1 NOP will add 4 T_{cy} making it $\frac{PW}{PS} = 6T_{cy}$ total making the frequency = 83.333kHz

• Evidence: Include or reference -

- the Lab 2 PORTB counter source flowchart
- the instruction cycle analysis of the repeating loop
- Predicted PW, PS, period, & frequency
- oscilloscope capture of the original RBO waveform
- measured PW, PS, period, & frequency
- calculated oscillator frequency
- comparison with the nominal 4MHz oscillator frequency
- modified source code showing the added NOP
- predicted and measured timing change caused by the NOP
- verification after the NOP is removed
- explanation of any difference between predicted and measured values.

- A waveform screen shot by its self is not sufficient. State what you expected, what was measured, and what the measurement demonstrates.

LG

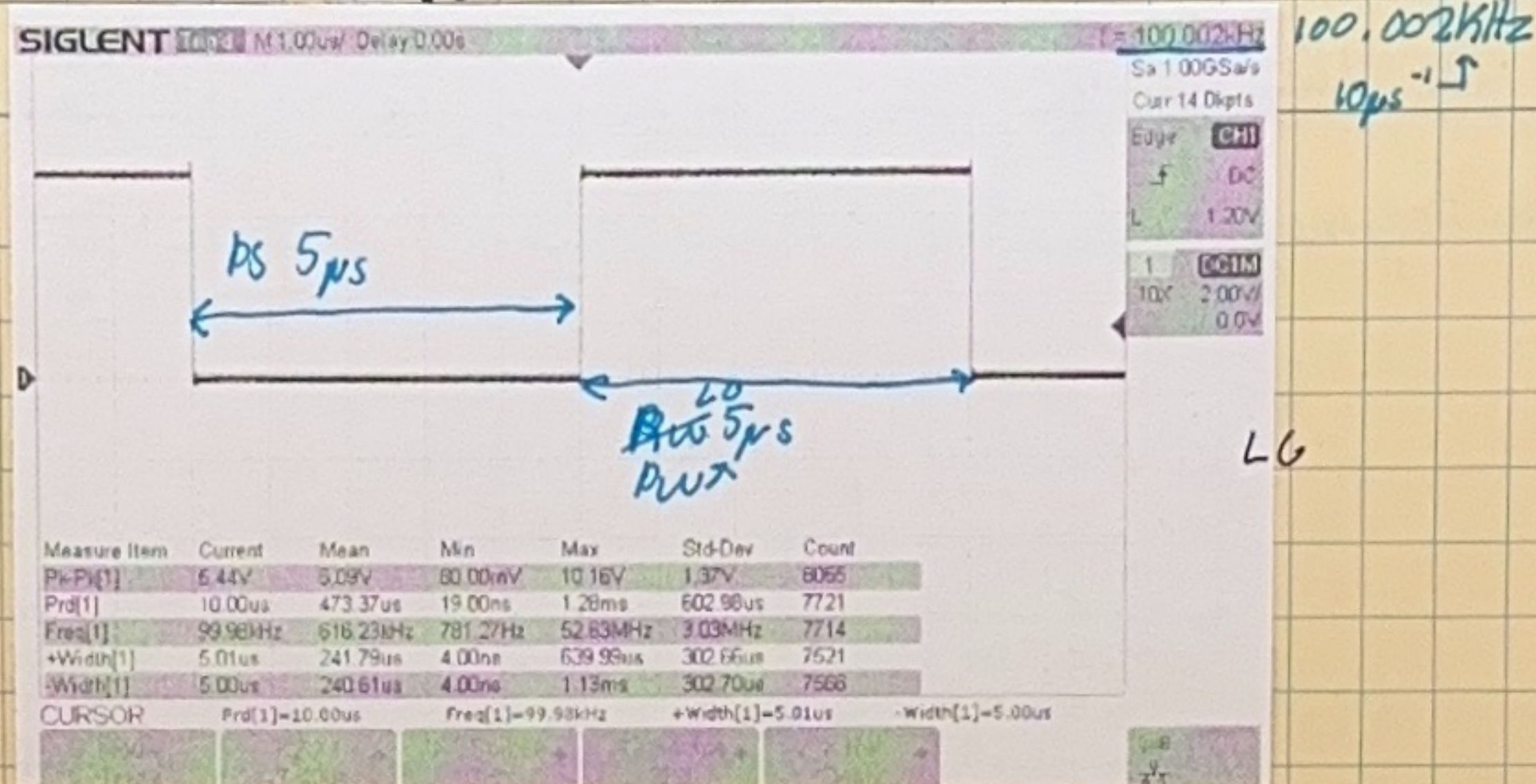


Figure 1: PORTB ABO incrementing output waveform @ 100kHz

```

Main:
; main code goes here

; Reset to Bank 0
BCF 0x03,6 ;set bank 0 - 1 Tcy
BCF 0x03,5 ;set bank 0 - 1 Tcy
INCF 0x06,1 ; increment PortB and place result in PortB - 1 Tcy

GOTO Main ; 2 Tcy
  
```

Figure 2: PORTB Incrementing output code (5Tcy) • Predicted - 5µs
• Measured - 5.01µs

This was the code used to increment portB so that the measured waveform in Figure 2. It has 5Tcy so its PW=5µs and PS=5µs with a period of 10µs.

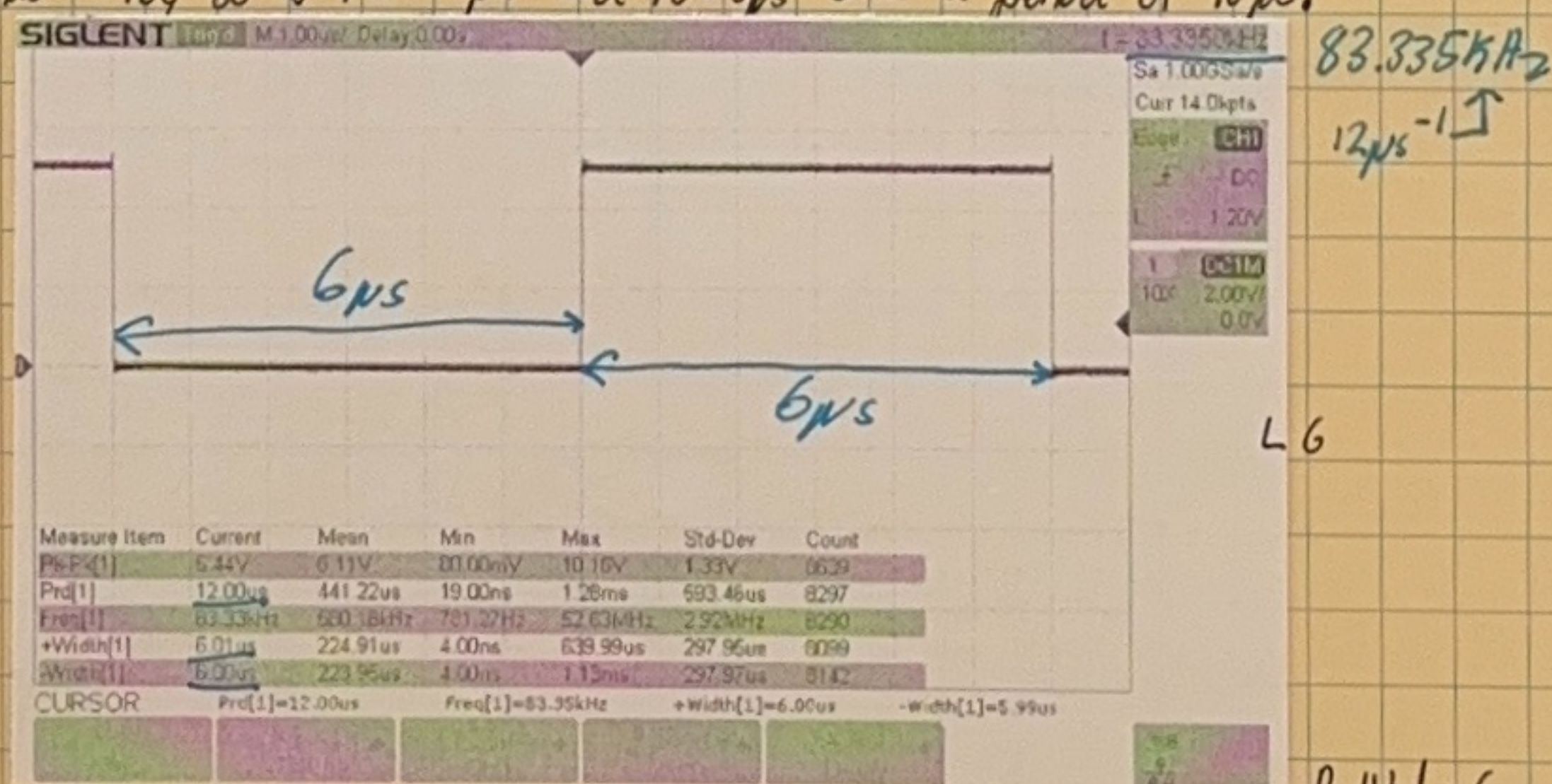


Figure 3: PORTB Incrementing with NOP included

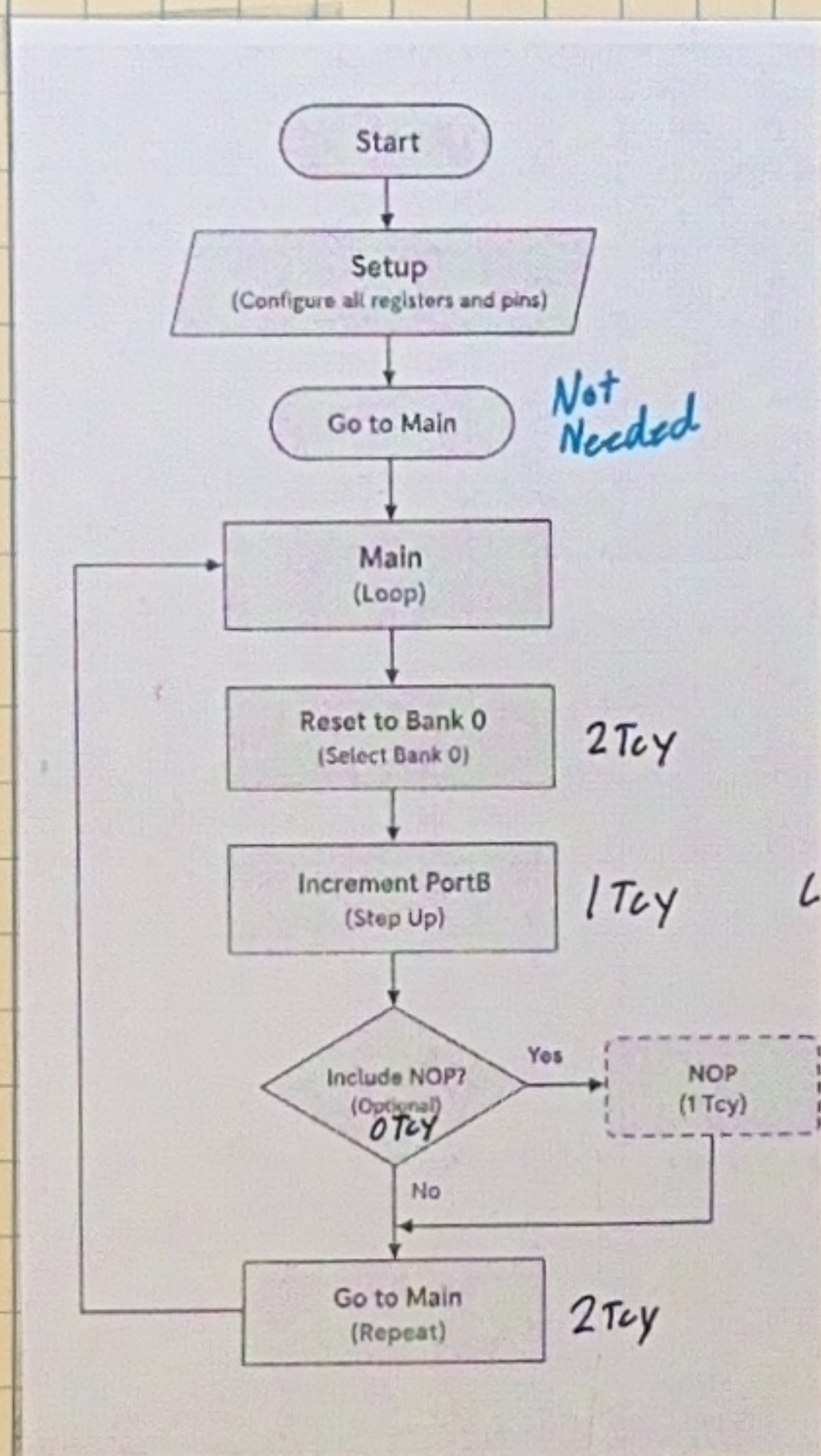
```

Main:
; main code goes here

; Reset to Bank 0
BCF 0x03,6 ;set bank 0 - 1 Tcy
BCF 0x03,5 ;set bank 0 - 1 Tcy
INCF 0x06,1 ; increment PortB and place result in PortB - 1 Tcy
NOP ; 1 Tcy
GOTO Main ; 2 Tcy
  
```

Figure 4: PORTB Incrementing with NOP included (6Tcy)

By adding the nop shown in figure 4 it adds one more T_{cy} which drops the frequency seen in Figure 3 but adds more time.



The flow chart shows the flow of the code. Showing the optional part of the NOP being included to change the Instruction Cycle Timing. Without NOP the $T_{cy} = 5\mu s$ and with $T_{cy} = 6\mu s$. The T_{cy} is the amount of time it takes for the PIC to execute a single basic instruction.

The SFR's for this part are in Lab 2 Part 2 pg 11 & 12 shows the banks used in setup.

After removing the NOP the frequency went back to normal.

This is a manual setting when done.

Figure 5: Flow chart configuration of code

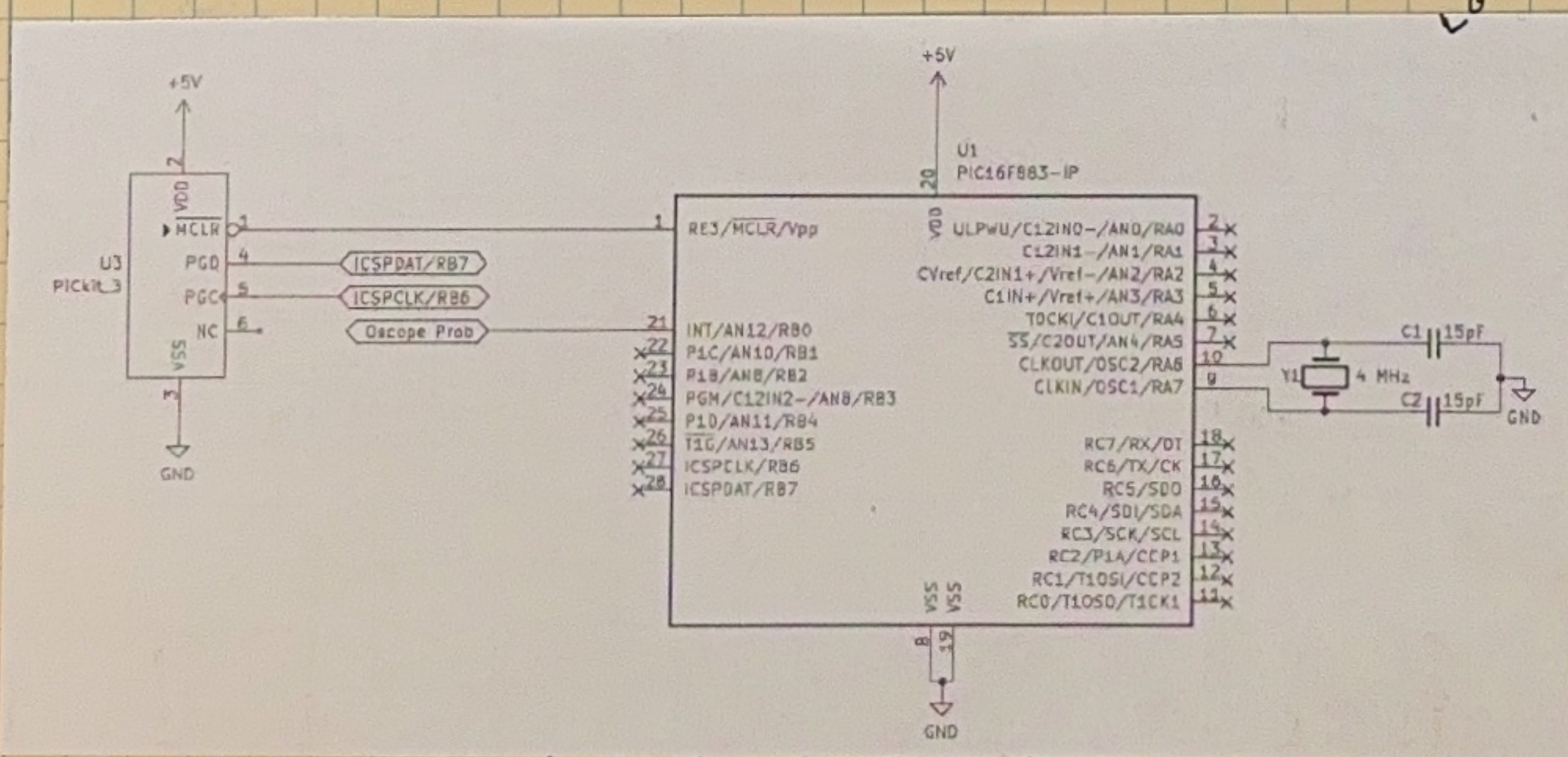


Figure 6: Schematic Schematic for Measuring PORTB (RBO)

Figure 6 just shows that (RBO) pin 21 is where the measurement captures were taken from. The PIC16F883-IP is removed after program is uploaded.

Verified working tree is clean and that the circuit works. RBO is the LSB and its count while incrementing is high the low but always equal. This is

Lab 3 - Delays

Part 2 - Create a Precise Short Delay

GOAL:

• Create a new PIC16F883 project that toggles RBO using XORWF, then add a calculated software delay to produce a 10KHz square wave.

• Use nominal instruction cycle time of $1T_{cy} = 1\mu s$ for all initial predictions and design calculations.

• After measuring the circuit, use the actual instruction cycle time determined in Part 1 to calculate the expected real timing and compare it with the measured results.

Before Lab:

• Create a new MPLAB project for this part - Use project structure, PIC configuration, oscillator configuration, and assembly convention in lab 2.

• Begin with this repeating output loop:

MainLoop:

MOVLW 0x01 ; 1T_{cy}

XORWF PORTB, 1 ; 1T_{cy}

GOTO MainLoop ; 2T_{cy}

- Determine what ~~what~~ waveform this program should produce on RBO.
 $1T_{cy} = 1\mu s$

Predict -

• PW = $4\mu s$

• PS = $4\mu s$

• period = $8\mu s$

• frequency = $125KHz$

• Duty cycle % = 50%

In the Lab:

1) Verify the XORWF toggle loop

- Build and program the new project using the unmodified loop. Measure RBO using the oscilloscope and frequency counter

Record:

• PW = $4.01\mu s$

• PS = $4.00\mu s$

• period = $8\mu s$

• frequency = $125,000.169Hz$ - frequency counter measurement

• Duty cycle = 50.125%

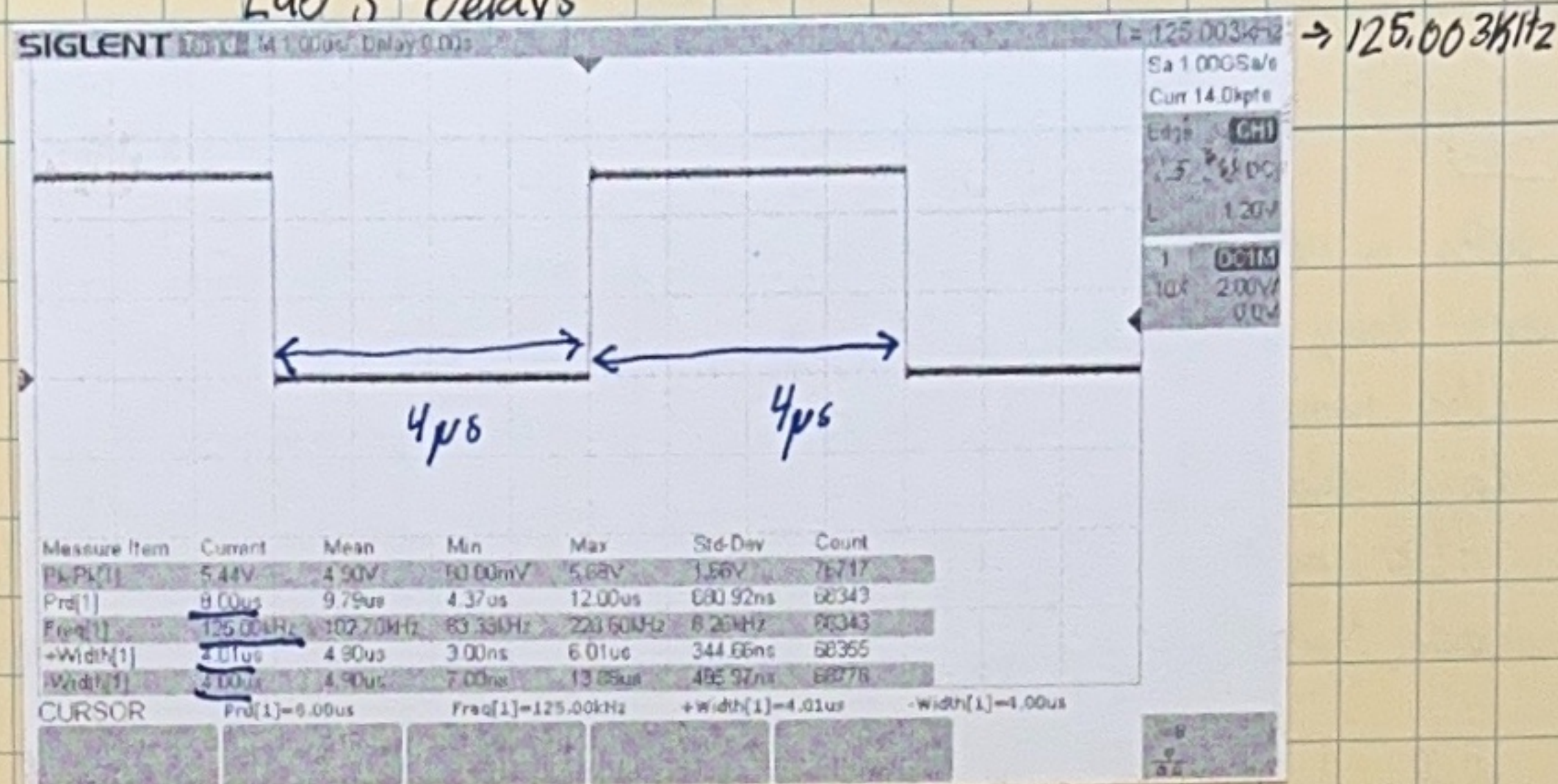
• See figure 7 for measurements Pg 32.

- Compare the measured vs predicted - Then use the actual T_{cy} measured in Part 1 to calculate the expected PW, PS, period, frequency, and duty cycle for the actual processor.

• Compare Explain why XORWF PORTB, 1 causes RBO to alternate between High and Low.

- This is because the XORWF acts as an inverter. The inverter switches state on the RBO setting it High or Low.

Lab 3 - Delays



• Figure 7: XORWF test code to show function - acts like invert see pg 31 for code.

2) Determine the required timing -

• Modify the program so RBO produces - 10kHz frequency & 50% duty cycle.

$$10\text{kHz} = \text{period of } 100\mu\text{s}$$

$$50\% \text{ duty cycle} = 50\mu\text{s} = \text{PS, PW}$$

3) Create a software delay loop -

• Use an 8-bit counter and DECFSZ to add the required delay

$$\text{equation for delay loop} \rightarrow ((N)(1+2)) - 1 = \text{desired delay} + \text{other } T_{cy} \text{ in loop}$$

4) Determine the counter value -

• Determine the required number of instruction cycles between RBO transitions.

$$\frac{3N-1}{+1 \quad +1} = 44 \rightarrow \frac{3N}{3} = \frac{45}{3} \rightarrow N = 15$$

5) Build and measure the calculated delay -

• Build and program the PIC16F883 using the calculated delay. Measure RBO using both the oscilloscope and frequency counter.

Evidence:

- timing calculation for the original XORWF loop using $1T_{cy} = 1\mu\text{s}$
- oscilloscope and frequency-counter measurements of the original XORWF loop
- expected timing of the original loop using the measured T_{cy} from Part 1
- calculation of the required 10kHz waveform
- calculation of the required counter value
- derivation of the delay-loop timing
- calculations for any NOP Instructions used
- modified source code showing the completed delay
- predicted PW, PS, period, frequency, and duty cycle using $1\mu\text{s}/T_{cy}$
- expected PW, PS, period, frequency, duty cycle using measured T_{cy}
- oscilloscope measurements
- frequency-counter measurements
- comparison of predicted, expected and measured values.
- explanation of any differences.
- same schematic used in Part 1 was used see figure 6 pg 30.

count EQU 0x20 ;use data memory address 0x20 as an 8-bit loop counter

Main:

MOVLW 0x01 ;1 Tcy increment by 1
XORWF 0x06,1 ;1 Tcy inverter and increment by 1

MOVLW 0x0F ;1 Tcy move W=15 to count
MOVWF count ;1 Tcy make count equal 15

DelayLoop: ; DelayLoop is equals 44 Tcy when count is 15
DECFSZ count,1 ;1 Tcy if not zero, 2 Tcy if zero
GOTO DelayLoop ;2 Tcy delay runs 15 times before skipping

GOTO Main ;2Tcy }main

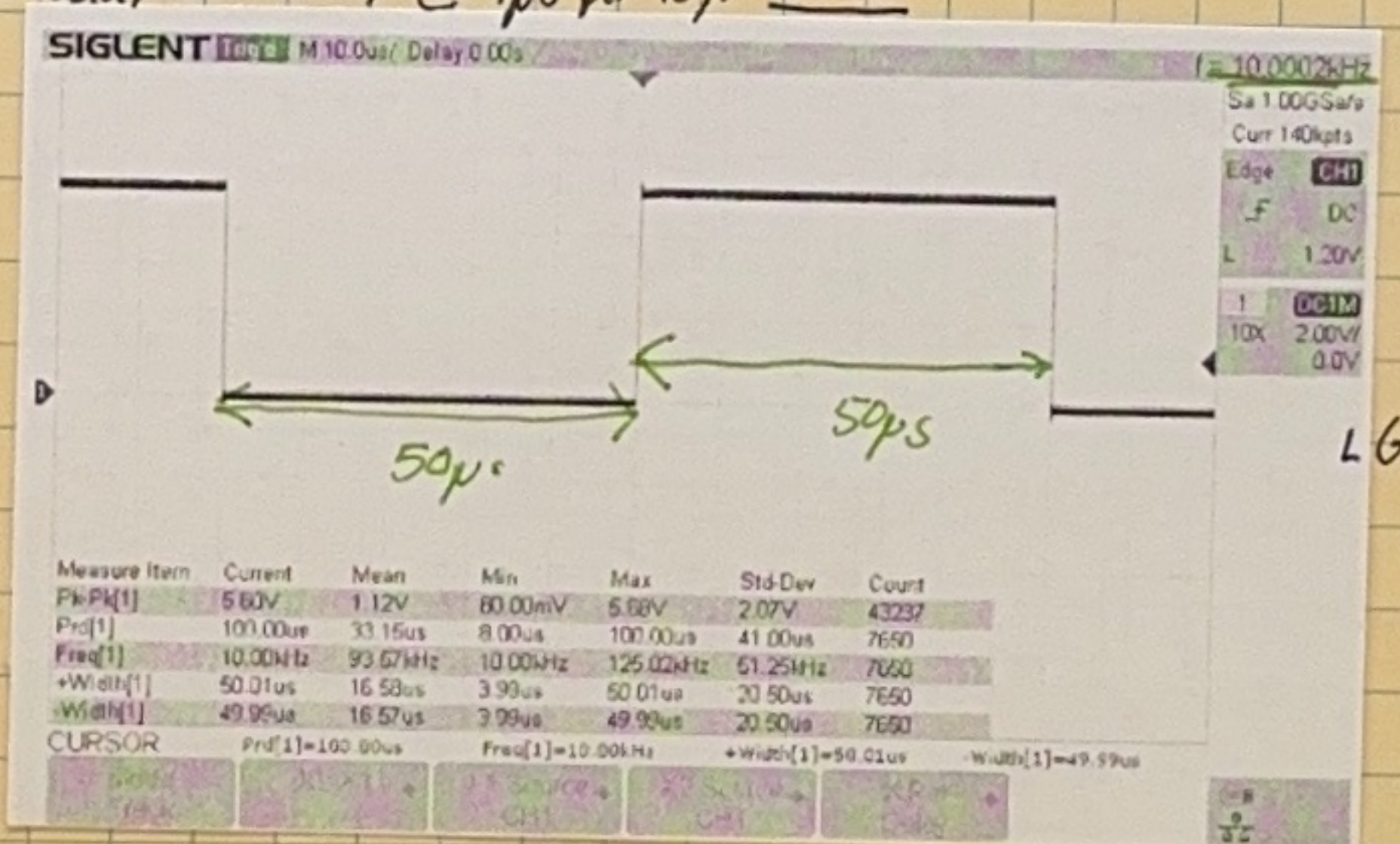
L6

Main

Delay

• Figure 8: Delay Loop code to make a 10KHz signal @ a period of 100µs

- The Main parts of the code was (6Tcy) so using the equation of $((N)(2+1))-1$ to get the total delay of 50Tcy @ 1µs per Tcy. $N=15$



• Figure 9: Capture of waveform @ 10KHz with a period of 100µs

- This is proof that using 15 for count gave the desired output wanted for a 10KHz signal on RBO.

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	50µs	50µs	50.01µs	50.005µs ^{L6}
PS	50µs	50µs	49.99µs	49.993µs ^{L6}
Period	100µs	100µs	100µs	99.977µs ^{L6}
Frequency	10KHz	10,000.00 ^{L6} 10KHz	10.002KHz	10,000.011 Hz
Duty cycle	50%	50%	50.01%	50.005% ^{L6}

• Table 1: Predicted vs Measured for delay/loop

- Demonstrate: the 10KHz RBO output using the oscilloscope and frequency counter.
- See pag 34 for flow chart showing the flow of the program.
- Working tree down and all files have been updated.
- ^{L6} Basatu The resolution of the oscilloscope isn't as good as the frequency counter.

L6

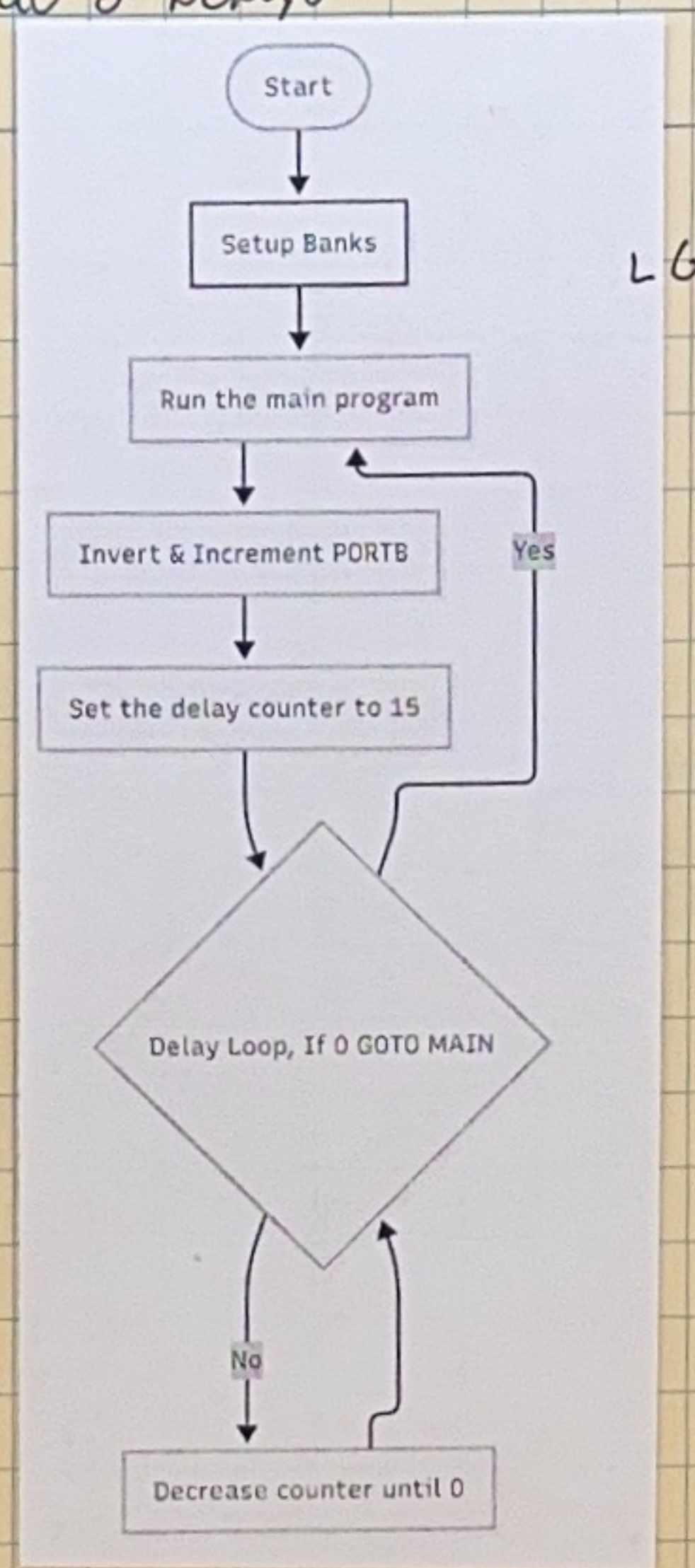


Figure 10: Flow chart of Delay^{LG} Delay Loop code

• Part 3 - Create and verify a long Delay

• Goal:

- Use nested software delay loops to create a much longer time delay^{LG} interval that can be produced conveniently with a signal 8-bit. Design a program that alternates the DLG 7137 display between 5 & 0 with a nominal pulse width and pulse space of:

$$PW = PS = 0.5s$$

- The timing calculation must be accurate to within one nominal instruction cycle:
 $\pm 1\mu s$

- Then determine whether the available measurement equipment can verify that level of timing accuracy:

• Before Lab: using $T_{cy} = 1\mu s$ for calculations

- Determine the longest delay that can be produced using the single 8 bit loop method in part 2 and explain why that method is not practical for creating a 0.5s delay

$$3(N) - 1 + 6 \Rightarrow 3(N) + 5 \rightarrow \text{Max for } N = 256 \rightarrow 3(256) + 5 = 773\mu s$$

which is nowhere close to being half a second^{LG} sec.

- Design an inline nested loop capable of producing the required interval. No using
• subroutines • timer peripherals • interrupts

- Account for every instruction executed between changes of the display value, including:

• initialization of each counter • inner loop execution • outer loop execution • skipped instructions.

• branch instructions • instructions used to change the display • Any Nop's for adjustments.

- Use a nominal instruction cycle time of $1\mu s$ to predict PW, PS, period, frequency, & duty cycle

- PW - $0.5s$ or $500ms$
- PS - $0.5s$ or $500ms$
- Period - $1.0s$ or $1000ms$
- Frequency - $1Hz$
- Duty cycle - 50%

• Measurement precision and resolution:

- Every measurement has a limit to the amount of detail the instrument can distinguish.
- When recording
 - record the measured value
 - record the displayed measurement resolution
 - consider whether that resolution is sufficient to support the conclusions made.

• In the Lab:

- 1) Implement the calculated nested delay. The PIC67137 must alternate continuously between (5, 0, 5, 0) with a nominal $0.5s$ interval between changes.
 - show modified source code containing the nested delay
- 2) Measure with the oscilloscope - use the PIC output signal that changes state at the same interval as the display value so the timing can be measured electrically.
- 3) Measure with the frequency counter - compare to oscilloscope and predicted
- 4) Change the program by one instruction cycle - add exactly one NOP to the execution path between output transitions. Before programming the PIC, predict the resulting change -

- $PW = 0.500001s$
- $PS = 0.500001s$
- period = $1.000002s$
- frequency = $1.000002Hz$

- this could be almost impossible to see on the oscilloscope but might be seen on the frequency counter.

- Remove the NOP, program the PIC again, and determine whether the measurement returns to its previous value

$$Inloop = 3N - 1$$

$$Outloop = 8(3N + 4) - 1 \quad \text{for overhead main}$$

$$Main = K[m(3n+4)+4]-1+6 \quad \text{solve for } 500,000 T_{cy}$$

$$K[100(3(500)+4)+4]-1+6 = 500,000$$

$$K(100(3(500)+4)+4) = 499,995$$

$$K(150,404) = 499,995$$

$$K = 3.324$$

$$N = 161$$

$$M = 5$$

$$K = 205$$

$$T_{cy} = 499,995 T_{cy}$$

This value give a T_{cy} of 451,212 which isn't close enough. So the math will be redone to solve for the best fit.

```

Main:
    MOVLW 0x01      ;1 Tcy increment by 1
    XORWF 0x06,1    ;1 Tcy inverter and increment by 1

    MOVLW 0x05      ;1 Tcy toggle the display 5 and 0
    XORWF 0x07,1    ;1 Tcy send 5 to ports to display

    MOVLW 0xCD      ;1 Tcy move W=205 to count
    MOVWF maincount ;1 Tcy make count equal 205

MainDelayLoop:
    MOVLW 0x05      ;1 Tcy move W=5 to count
    MOVWF outcount  ;1 Tcy make count equal 5

OutDelayLoop:
    MOVLW 0xA1      ;1 Tcy move W=161 to count
    MOVWF incount    ;1 Tcy make count equal 161

InDelayLoop:
    DECFSZ incount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO InDelayLoop ;2 Tcy delay runs 161 times before skipping

    DECFSZ outcount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO OutDelayLoop ;2 Tcy delay runs 5 times before skipping

    DECFSZ maincount,1 ;1 Tcy if not zero, 2 Tcy if zero
    Goto MainDelayLoop ;2 Tcy delay runs 205 times before skipping

GOTO Main          ;2 Tcy

```

Main count = 205 or CD
 Out count = 5 or 05
 In count = 161 or A1

Figure 11: Main code with the 3 delays nested test Not correct

- The full equation looks like this - $205(5(3(161)+4)+4)+8 = 500,002 \text{ Tcy}$
 - There was a lot of different values that can be used to solve for this and this is currently outside tolerance so below is the exact values used to get as close as possible.

$$10(99((3(167)+4)+4)+8)+2 = 500,000 \text{ Tcy}$$

```

Main:
    MOVLW 0x01      ;1 Tcy increment by 1
    XORWF 0x06,1    ;1 Tcy inverter and increment by 1

    MOVLW 0x05      ;1 Tcy toggle the display 5 and 0
    XORWF 0x07,1    ;1 Tcy send 5 to ports to display

    MOVLW 0x0A      ;1 Tcy move W=10 to count
    MOVWF maincount ;1 Tcy make count equal 10
    NOP             ;1 Tcy
    NOP             ;1 Tcy
    NOP             ;1 Tcy

MainDelayLoop:
    MOVLW 0x63      ;1 Tcy move W=99 to count
    MOVWF outcount  ;1 Tcy make count equal 99

OutDelayLoop:
    MOVLW 0xA7      ;1 Tcy move W=167 to count
    MOVWF incount    ;1 Tcy make count equal 167

InDelayLoop:
    DECFSZ incount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO InDelayLoop ;2 Tcy delay runs 167 times

    DECFSZ outcount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO OutDelayLoop ;2 Tcy delay runs 99 times before skipping

    DECFSZ maincount,1 ;1 Tcy if not zero, 2 Tcy if zero
    Goto MainDelayLoop ;2 Tcy delay runs 10 times before skipping

GOTO Main          ;2 Tcy

```

Main count = 10
 Out count = 99
 In count = 167

Figure 12: Code showing the cycle delay of 500,000 Tcy

- This is the calculated code to give the value of 500,000 Tcy which @ 1µs per Tcy gives us the 0.5s with only 2 NOP's.

9/9/2026
LR6

Lab 3 - Delays

37

Setup:
; add device setup here

```

; Bank 0
BCF 0x03,6 ;Set Bank 0
BCF 0x03,5 ;Set Bank 0
CLRF 0x06 ;Setting PortB on reset low
MOVLW 0x30 ;Load 0 for start up
MOVWF 0x07 ;Set PortC to display 0

; Bank 1
BCF 0x03,6 ;Set Bank 1
BSF 0x03,5 ;Set Bank 1
CLRF 0x86 ;Set PortB as an output TRISB
CLRF 0x87 ;Set PortC as an Output TRISC

; Bank 2
BSF 0x03,6 ;Set Bank 2
BCF 0x03,5 ;Set Bank 2
BSF 0x09,1 ;Setting Bit 1 High for CM2CON1

; Bank 3
BSF 0x03,6 ;Set Bank 3
BSF 0x03,5 ;Set Bank 3
CLRF 0x89 ;Setting the I/O to digital ANSELN

;Reset to Bank 0
BCF 0x03,6 ;Set Bank 0
BCF 0x03,5 ;Set Bank 0

GOTO Main

maincount EQU 0x20 ;use data memory address 0x20 as an 8-bit loop counter
outcount EQU 0x21 ;use data memory address 0x21 as an 8-bit loop counter
incount EQU 0x22 ;use data memory address 0x22 as an 8-bit loop counter

```

LG

Figure 13: Setup code and setting the memory address.

- Code made to Setup the ports and set the count for 0 and 5 to toggle every cycle.

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	500,000 μ s	500,000 μ s	499.99ms	0.4999999990 NA ^{LG}
PS	500,000 μ s	500,000 μ s	499.99ms	- NA
Period	1.0s	1.0s	999.98ms	- NA
Frequency	1 Hz	1 Hz	1.00Hz	- NA
Duty cycle	50%	50%	50%	- NA

Table 2: Predicted vs^{LG} Measured values for 500,000TCY

- frequency counter jumps around to much and will not give an recordable memorje.

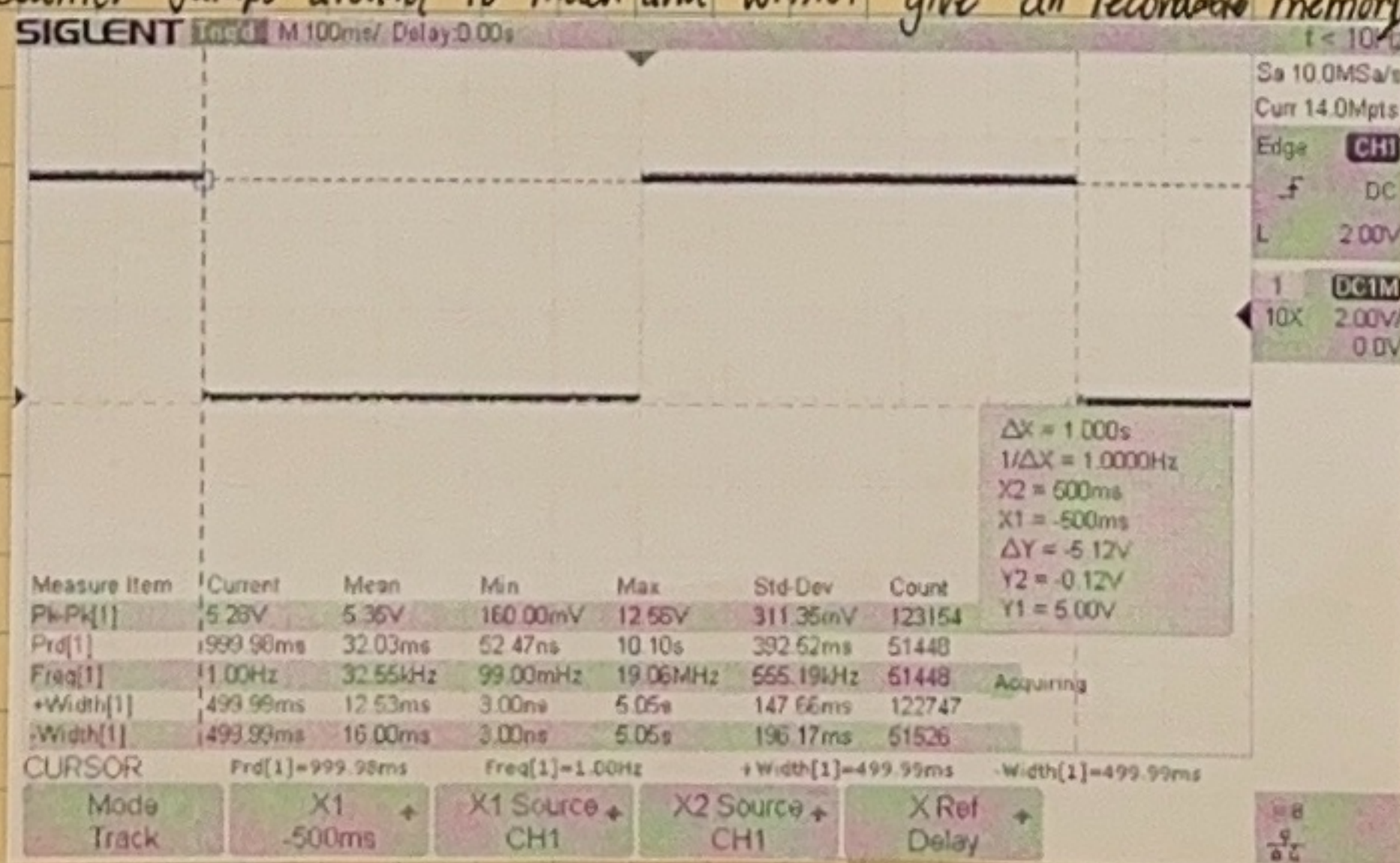


Figure 14: Measured Period with cursors and measuring tool.

- This capture shows the measurements that display the 0.5s desired. The oscscope has ~~lots~~^{lots} of room of error and the cursors can only read as well as the eye can see. The frequency counter still bounces to much, to read ~~any~~^{any} thing.

9/9/2026
LR6

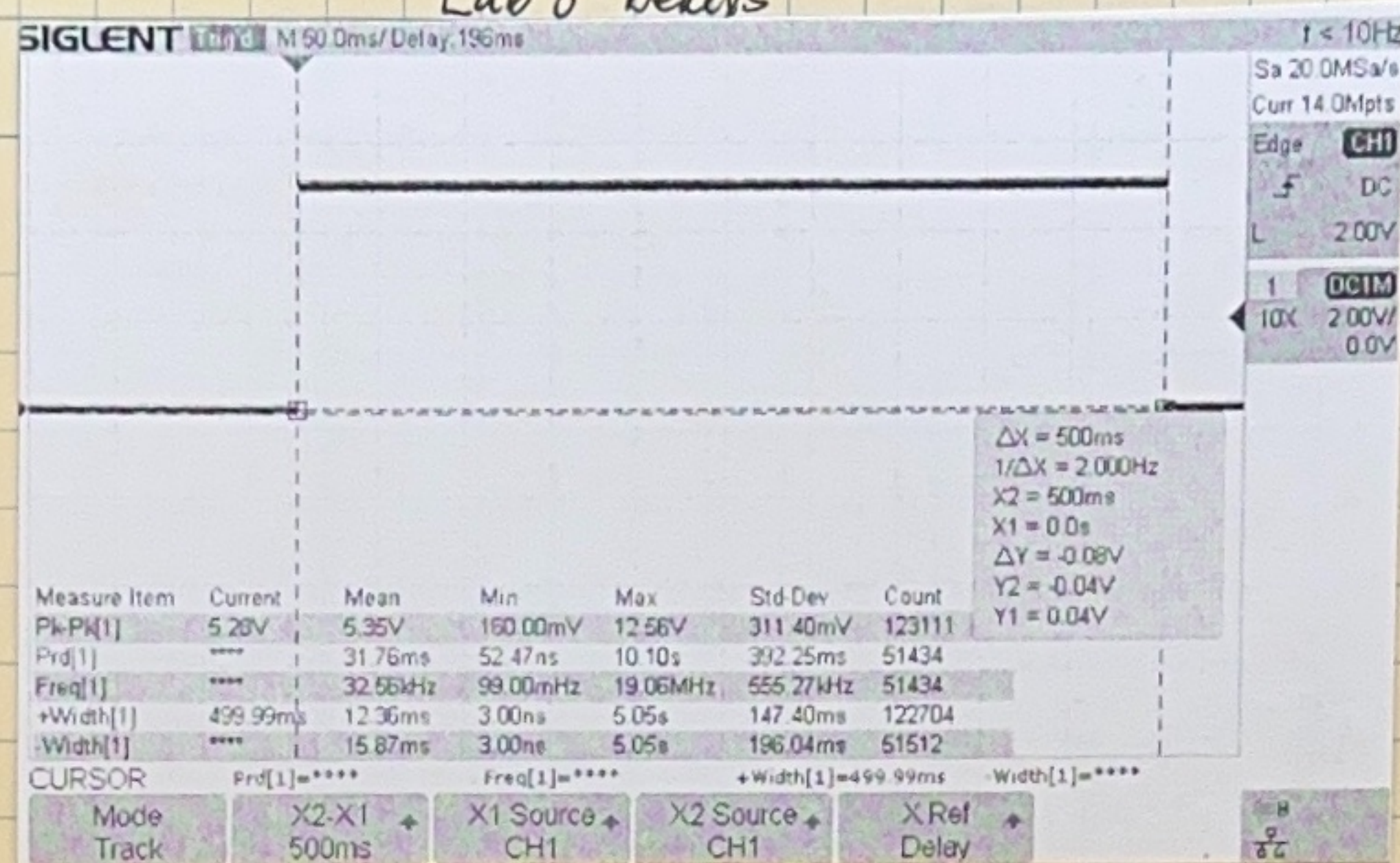


Figure 15: Close up of PW and this was done on the add one NOP

- even after adding a nop there was no real ^{L6} readable differences using the oscilloscope. the oscilloscope can only read up to as well as the eye can see. Still frequency counter doesn't work at this range. The frequency counter works after setting the voltage limit and DC but PW is the only reading

Measurement	Before NOP	With NOP	Difference	Instrument Resolution
PW	^{L6} 500ms 499.99ms 0.499999900	0.500000909 499.99ms	1.008ps	50ms per division

Table 3: Test and Measured with and without a NOP ^{L6} NOP

- This table shows the measured value of the code with and without NOPs added to the 3 in the code.

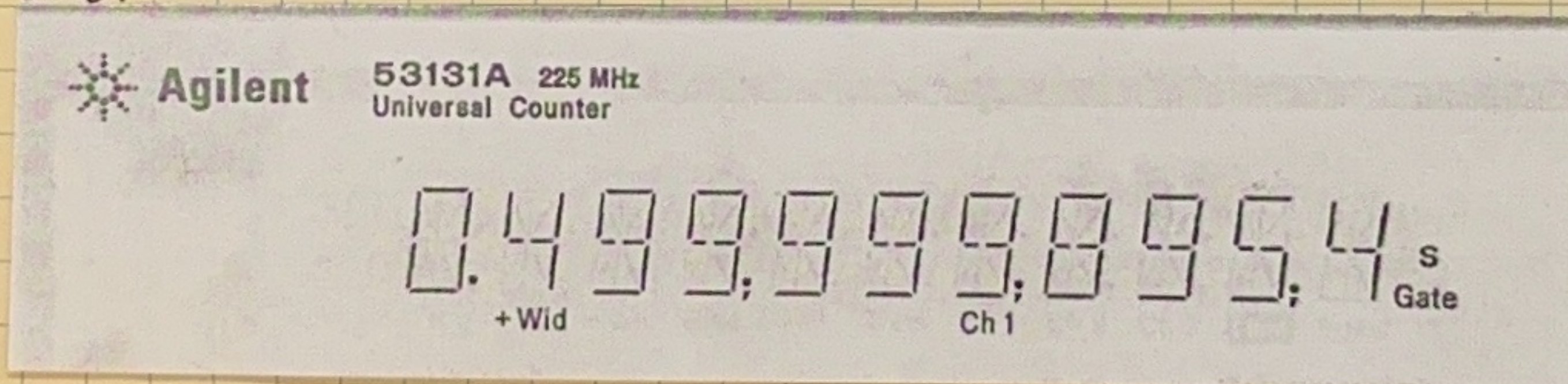
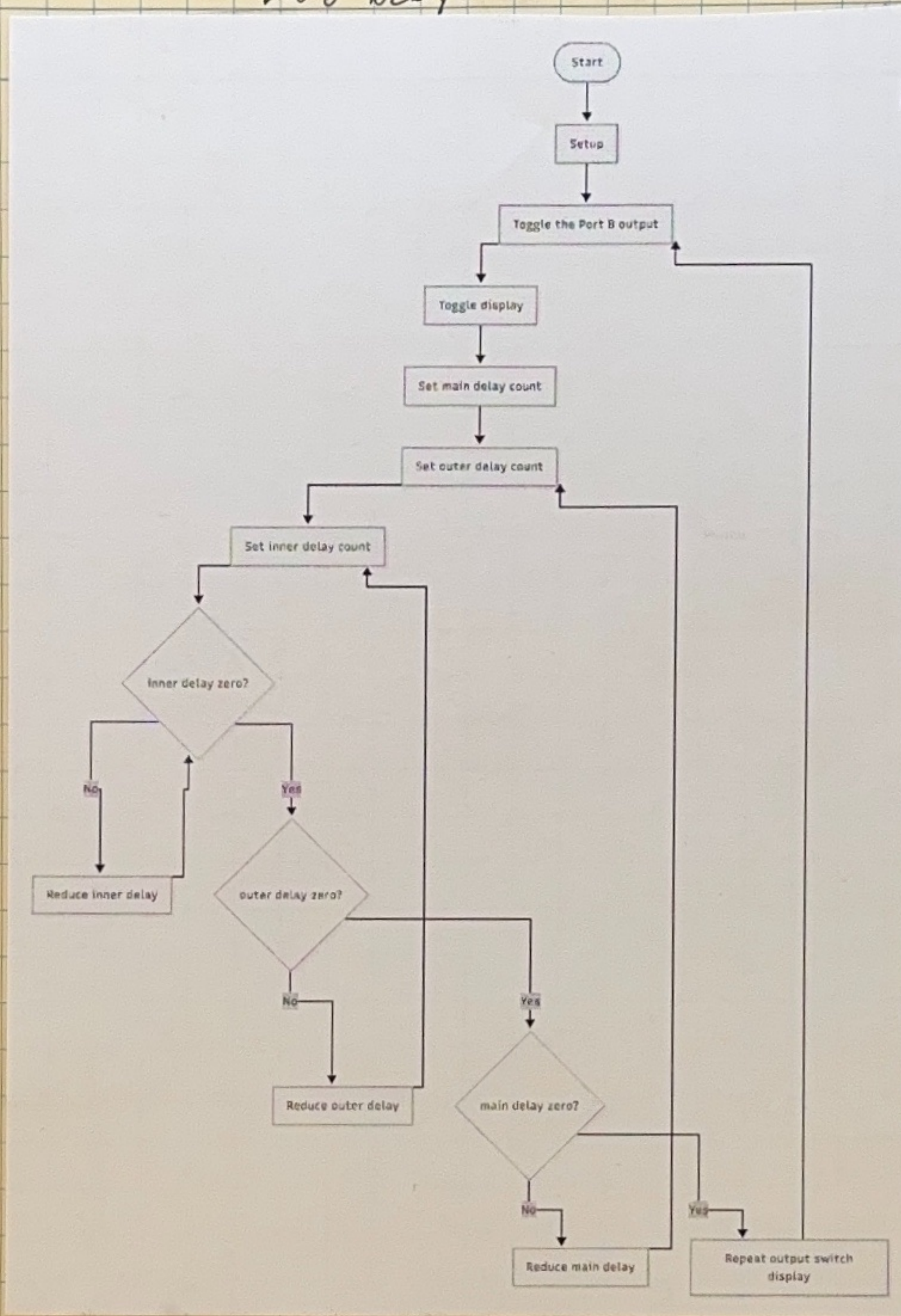


Figure 16: Measured PW from the Frequency counter

Demonstrate:

- explain why nested loops ^{L6} are required
- explain the timing calculation from the innermost loop outward
- show that the nominal design is within an instruction cycle of the requested 0.5s interval
- state the resolution of the oscillator ^{L6} measurement
- measure PW using the oscilloscope
- explain whether the oscilloscope measurement alone can verify +/- 1 using timing accuracy
- compare the 2 measurements methods
- measure PW using the frequency counter
- predict the effect of adding one NOP
- add or remove a NOP when instructed.
- Show the DLG7137 switching between 0 & 5

^{L6}



Flow^{LG} Figure 17: Flow chart of the process.

• Here the oscilloscope was easier to use to see the big picture but the frequency counter it very percies it just took a lot of adjusting settings in order to get it to work accordingly. It's very tempermental and needs some small adjustments to work as desired. It's only really able to read PW everything else tried messes things up. So both instruments are need but the counter will ~~per~~^{LG} read all the way to pico sec where the oscilloscope only reads to milla sec.

TJR
9-10-26

Part 4 - Delay Subroutines and the Hardware Stack

- Goal: Refactor the working inline delay from Part 3 into reusable subroutines.
- Use CALL & RETURN, including a nested subroutine call and verify how the new program structure affects execution timing and the hardware return stack.

Before Lab: Modify the Part 3 program so it contains:

- a main program that alternates the DLG7137 between 5 & 0.
- a reusable long delay subroutine
- a shorter delay subroutine called from within the long - delay subroutine.
- appropriate CALL and RETURN instructions
- Create an updated flowchart showing the subroutine calls and return paths
- Recalculate the timing of the program.

- Include the execution time of

- Call
- Return
- delay Loop instructions
- Counter initialization
- display changes
- branches
- any timing-adjustment NOP instruction

- $T_{CY} = 1\mu s$ for predictions
- $PW = PS = 0.5s$

In the Lab:

- 1) Implement the subroutines - Build and program the refactored code
verify that the DLG7137 continuously alternates 5, 0, 5, 0...
- 2) Verify the timing - Measure using the oscilloscope & frequency counter.
 - PW
 - PS
 - period
 - frequency
 - duty cycle
 - compare the measurements with: the nominal prediction using $1\mu s/T_{CY}$ and the expected timing using measured T_{CY} from part 1.
- 3) Trace the nested calls: using the actual source code, trace one complete sequence -

Main

→ Call long delay

→ Call short delay

← Return

← Return

Main continues

- Identify: where each CALL transfers execution

 - what return address must be stored
 - the order in which Return uses those addresses
 - the order in which return addresses are placed on the hardware
 - when execution resumes after each Return.

9/9/2026
LRG

41

Lab 3 - Delays

- calculate call/Return loops

$$M = 74$$

$$K = 23$$

$$N = 89$$

$$\text{short delay} \rightarrow 2 + (N(2+1)-1) + 2$$

$$2 + (3N-1) + 2$$

$$\rightarrow 3N+3$$

$$\text{Outer Long Delay loop} \rightarrow \Delta(K(3N+3)(2+1)-1)((2+1)-1)$$

$$2 + (3N+3) + 1 + 2 \rightarrow (3N+8)$$

$$(M(K(3N+8)) + 4M + 3) + 6 = 500,000 Tcy$$

$$\begin{matrix} \uparrow & \uparrow & \uparrow & \uparrow \\ 74 & 23 & 89 & 74 \end{matrix}$$

```

Main:
    MOVLW 0x05      ;1 Tcy toggle the display 5 and 0
    XORWF 0x07,1    ;1 Tcy send 5 to PortC to display

    CALL LongDelay  ;2 Tcy CALL does the whole delay, then returns here

    GOTO Main       ;2 Tcy

LongDelay:
    MOVLW 0x4F      ;1 Tcy move W 79 to count
    MOVWF maincount ;1 Tcy make count 79

MainDelayLoop:
    MOVLW 0x17      ;1 Tcy move W 23 to count
    MOVWF outcount  ;1 Tcy make count 23

OutDelayLoop:
    CALL ShortDelay ;2 Tcy CALL, runs ShortDelay, then returns here

    DECFSZ outcount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO OutDelayLoop ;2 Tcy runs 23 times before skipping

    DECFSZ maincount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO MainDelayLoop ;2 Tcy runs 79 times before skipping

    RETURN          ;2 Tcy back to Main

ShortDelay:
    MOVLW 0x59      ;1 Tcy move W 89 to count
    MOVWF incount   ;1 Tcy make count 89

InDelayLoop:
    DECFSZ incount,1 ;1 Tcy if not zero, 2 Tcy if zero
    GOTO InDelayLoop ;2 Tcy runs 89 times before skipping

    RETURN          ;2 Tcy back to OutDelayLoop
    
```

• Help from ai to find numbers after many trys and after code was already written.

Figure 18: Main code for call & Return delay

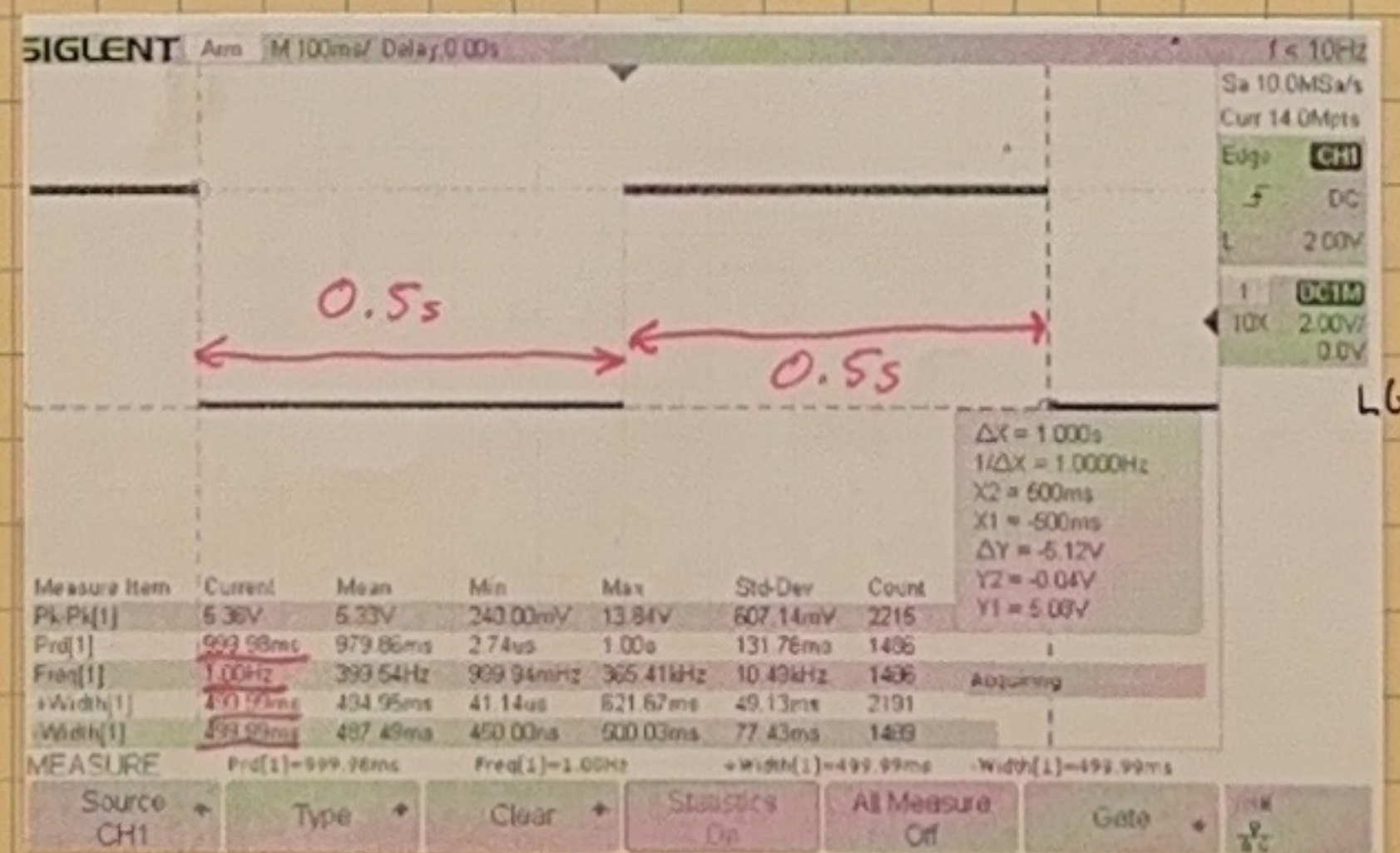


Figure 19: Capture of full period with Proof of 1Hz frequency

- The frequency counter said measurement for PW=0.500,000,004,9 which is very very accurate. I trust the counters reading more then the oscopes.

9/10/2026

9/10/2026
LRG

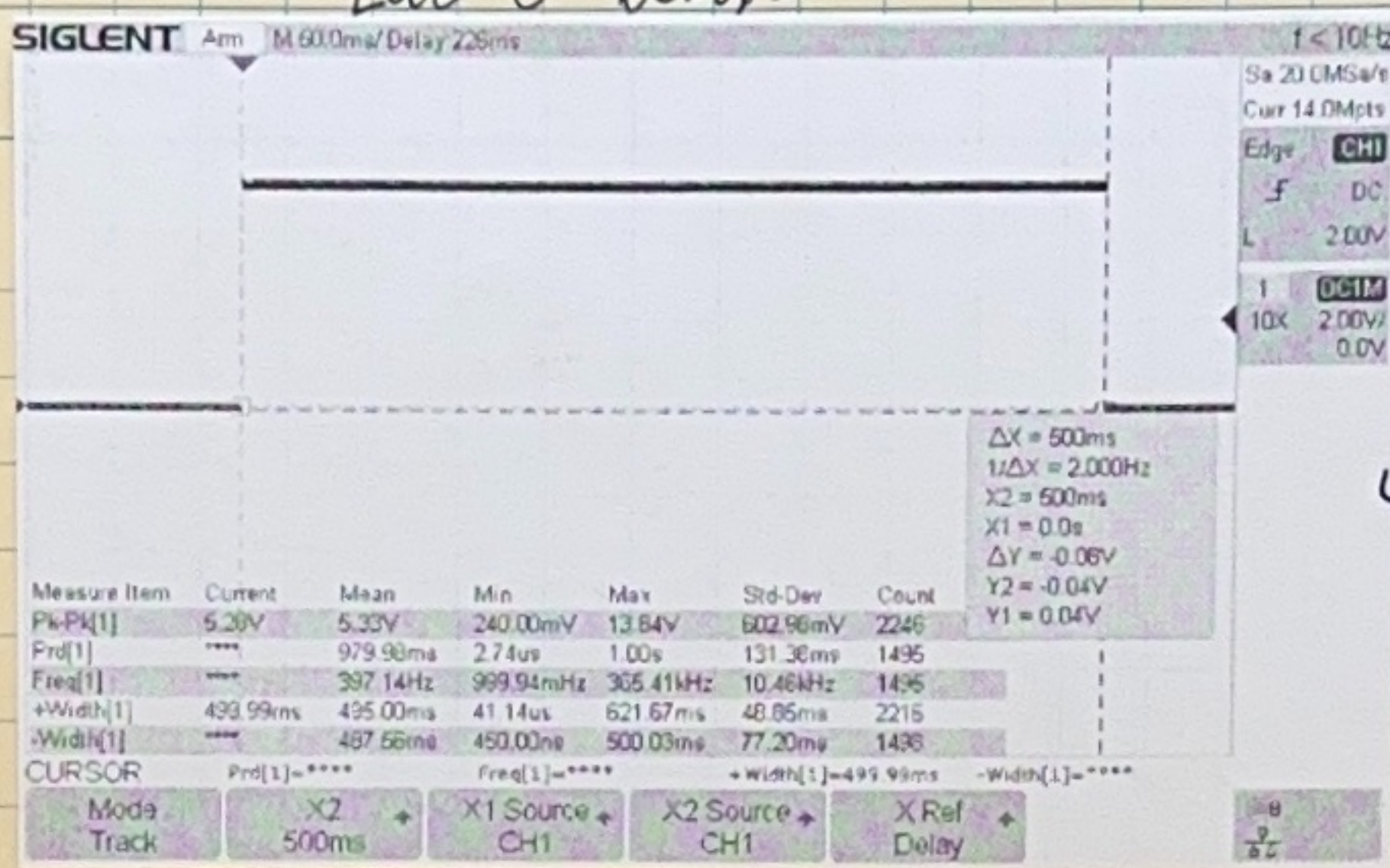


Figure 20: Close Capture of PW for clear detail of rise and fall edge

At the resolution of 50ms per division on the oscilloscope with the cursors only able to move 1ms at a time. Because of this the frequency counter gives the best reading on PW. Below are the measured values.

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	0.5 s	0.5 s	0.5 s	0.499999864
PS	0.5 s	0.5 s	0.5 s	- NA
Period	1.0 s	1.0 s	1.0 s	- NA
Frequency	1 Hz	1 Hz	1 Hz	- NA
Duty cycle	50%	50%	50%	- NA

Table 4: Predicted and measured values of TCY

Measurement	Before NOP	With NOP	Difference	Instrument Resolution
PW	0.499999864 s	0.500000984 s	11.2 ^{μs} 1.012 μs	10 n freq counter 50ms per division scope

Table 5: NOP Differences when added to code

These table show the measured values of using both the oscilloscope and the frequency counter. timing comparison with Part 3 is very very close see pg 38 for NOP measurements.

Evidence:

- Part 3 sources and calculations - see pg 36 & 37
- updated flow chart - see pg 43
- modified source showing the subroutines - see pg 41
- updated timing calculation with call and return - see pg 41
- trace of the nested calls and returns ^{μs} address - see pg 41
- predicted timing using lps/TCY
- expected timing using measured TCY
- oscilloscope measurements
- frequency-counter measurements
- comparison with Part 3 timing

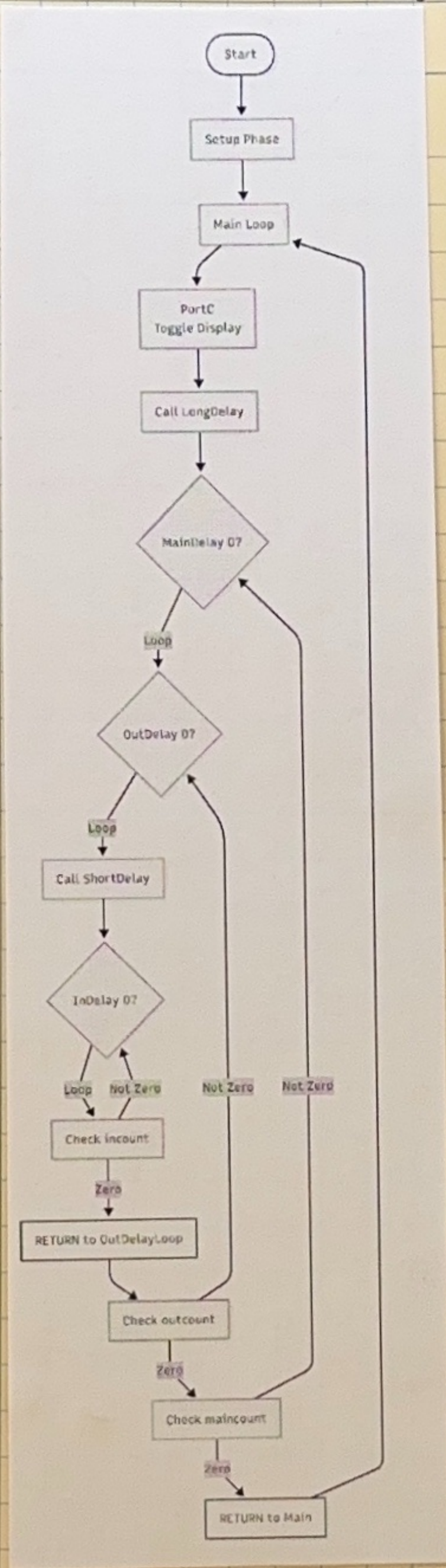


Figure 21: Flow chart showing the change of adding call/return

So when a call happens it pushes into the stack level like a pezz dispenser as a call happens it puts the newest call in and pushes the older one down and as it returns it POP's the newest call & out of stack.

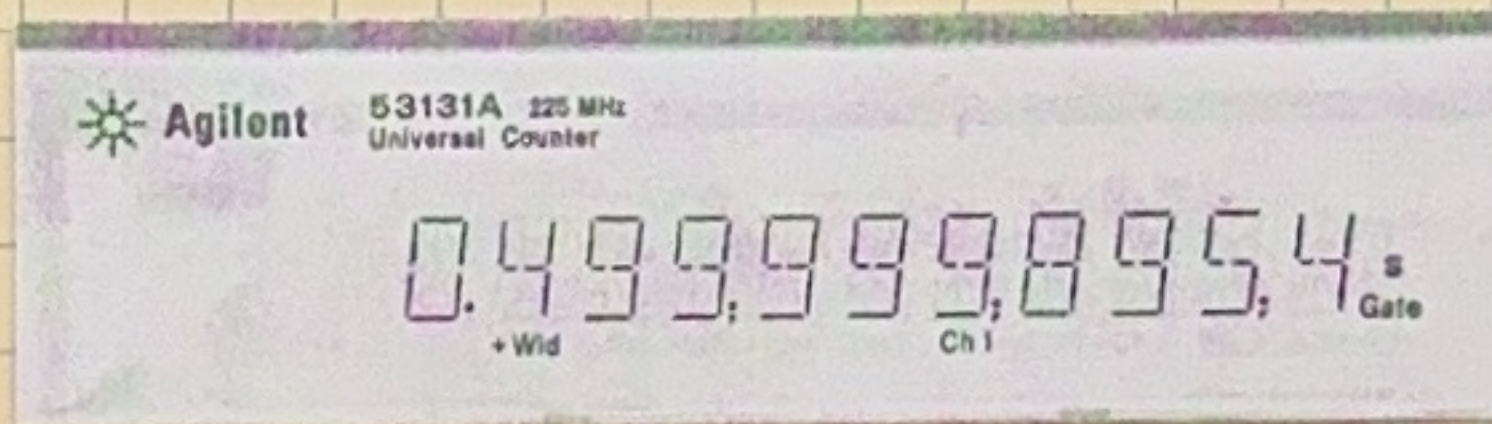


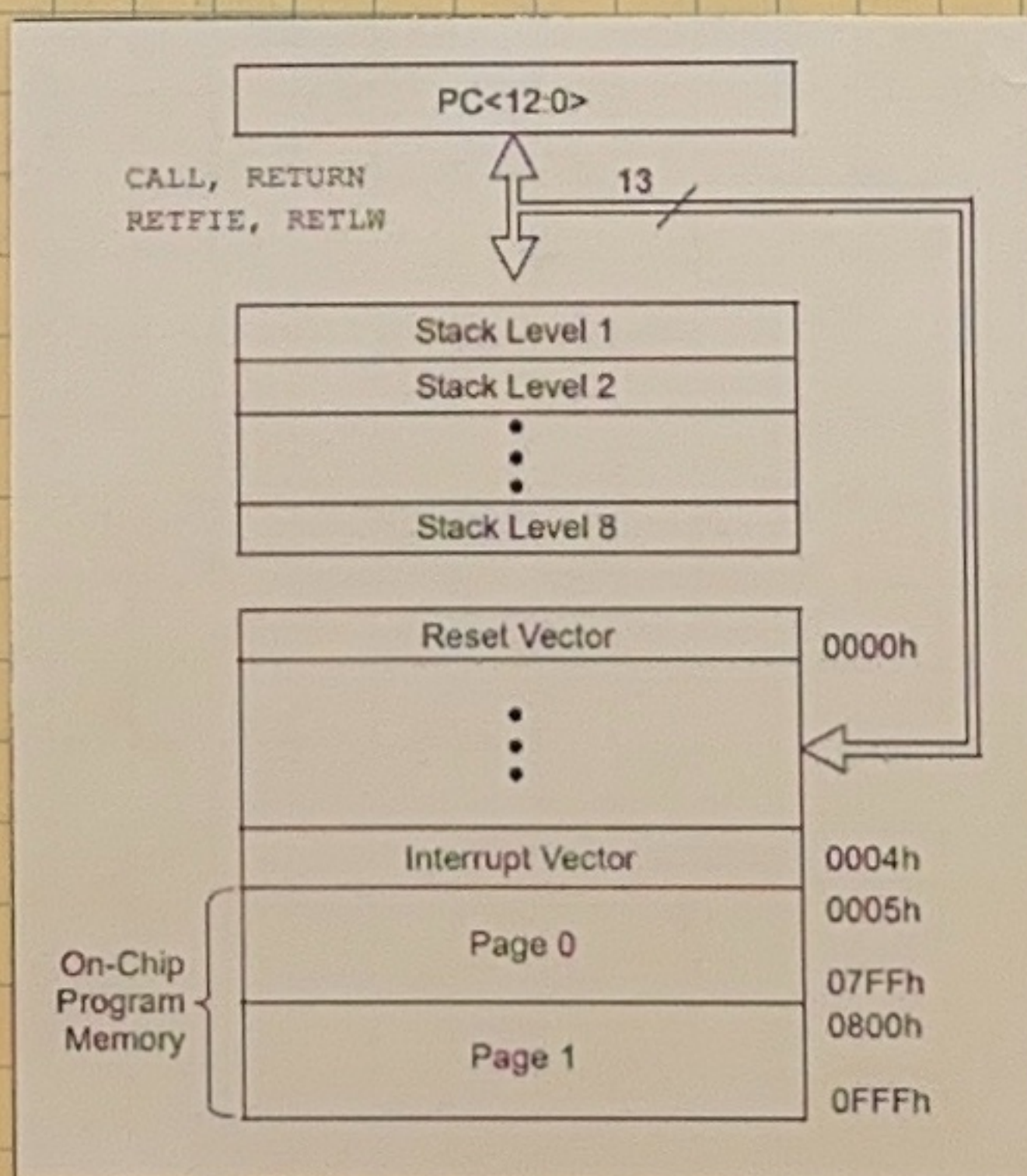
Figure 22: Counter without NOP



Figure 23: Counter with NOP

The Figures above show just how far the reading of the frequency counter can see for PW. The point is seeing that some instruments are better for precision reading vs overall.

The call wings the push and pop of the stack level in the Figure below which is from D#810001291 H 2-2 Program memory map of the PIC16F883.



TJR
9-10-26

Figure 24: Program Memory Map

Mnemonic, Operands	Description	Cycles	14-Bit Opcode		Status Affected	Notes
			MSb	LSb		
BYTE-ORIENTED FILE REGISTER OPERATIONS						
ADDWF	f, d	Add W and f	1	00 0111 dfff ffff	C, DC, Z	1, 2
ANDWF	f, d	AND W with f	1	00 0101 dfff ffff	Z	1, 2
<u>CLRF</u>	f	Clear f	<u>1</u>	00 0001 1fff ffff	Z	2
<u>CLRW</u>	-	Clear W	<u>1</u>	00 0001 0xxx xxxx	Z	
COMF	f, d	Complement f	1	00 1001 dfff ffff	Z	1, 2
DECf	f, d	Decrement f	1	00 0011 dfff ffff	Z	1, 2
<u>DECFSZ</u>	f, d	Decrement f, Skip if 0	<u>1(2)</u>	00 1011 dfff ffff		1, 2, 3
INCF	f, d	Increment f	1	00 1010 dfff ffff	Z	1, 2
<u>INCFSZ</u>	f, d	Increment f, Skip if 0	<u>1(2)</u>	00 1111 dfff ffff		1, 2, 3
IORWF	f, d	Inclusive OR W with f	1	00 0100 dfff ffff	Z	1, 2
MOVF	f, d	Move f	1	00 1000 dfff ffff	Z	1, 2
<u>MOVWF</u>	f	Move W to f	<u>1</u>	00 0000 1fff ffff		
<u>NOP</u>	-	No Operation	<u>1</u>	00 0000 0xx0 0000		
RLF	f, d	Rotate Left f through Carry	1	00 1101 dfff ffff	C	1, 2
RRF	f, d	Rotate Right f through Carry	1	00 1100 dfff ffff	C	1, 2
SUBWF	f, d	Subtract W from f	1	00 0010 dfff ffff	C, DC, Z	1, 2
SWAPF	f, d	Swap nibbles in f	1	00 1110 dfff ffff		1, 2
<u>XORWF</u>	f, d	Exclusive OR W with f	<u>1</u>	00 0110 dfff ffff	Z	1, 2
BIT-ORIENTED FILE REGISTER OPERATIONS						
<u>BCF</u>	f, b	Bit Clear f	<u>1</u>	01 00bb bfff ffff		1, 2
<u>BSF</u>	f, b	Bit Set f	<u>1</u>	01 01bb bfff ffff		1, 2
BTFSC	f, b	Bit Test f, Skip if Clear	1 (2)	01 10bb bfff ffff		3
BTFSS	f, b	Bit Test f, Skip if Set	1 (2)	01 11bb bfff ffff		3
LITERAL AND CONTROL OPERATIONS						
ADDLW	k	Add literal and W	1	11 111x kkkk kkkk	C, DC, Z	
ANDLW	k	AND literal with W	1	11 1001 kkkk kkkk	Z	
<u>CALL</u>	k	Call Subroutine	<u>2</u>	10 0kkk kkkk kkkk		
CLRWD	-	Clear Watchdog Timer	1	00 0000 0110 0100	$\overline{TO}$, $\overline{PD}$	
<u>GOTO</u>	k	Go to address	<u>2</u>	10 1kkk kkkk kkkk		
IORLW	k	Inclusive OR literal with W	1	11 1000 kkkk kkkk	Z	
<u>MOVLW</u>	k	Move literal to W	<u>1</u>	11 00xx kkkk kkkk		
RETFIE	-	Return from interrupt	2	00 0000 0000 1001		
RETLW	k	Return with literal in W	2	11 01xx kkkk kkkk		
<u>RETURN</u>	-	Return from Subroutine	<u>2</u>	00 0000 0000 1000		
SLEEP	-	Go into Standby mode	1	00 0000 0110 0011	$\overline{TO}$, $\overline{PD}$	
SUBLW	k	Subtract w from literal	1	11 110x kkkk kkkk	C, DC, Z	
XORLW	k	Exclusive OR literal with W	1	11 1010 kkkk kkkk	Z	

Figure 25: Instruction Set for PIC16F883 Doc# DS40001291H pg 227

- The underlined instructions were used throughout the lab but this ^{LG} is in to show the Tcy of each instructions and how they can be used and how the equations were solved.

Conclusion -

Lab 3 was all about how to code a delay and calculate an exact delay. Delays make it so the program takes an exact amount of time to happen. The first part of the lab was finding our measured Tcy of an instruction. Using the frequency counter to try to get as the best measurement possible. (1.0012ps) Then taking that information, the next step is making a short precise delay of a period of 100ps and a PW/PS be 50ps. Both provided and measured with oscilloscope and frequency counter to prove precision as well as see how to best resolution. All this was because the next parts were making a code that had a Tcy of 500,000 or a 0.5s for PW and PS. The purpose was to be as ^{LG} precise as in $\pm 1ps$. So by using nested loops to create a looping effect. Using 3 loops to make enough loop in the 8bit register. The last part was using a call and return to recreate the same code but by using call to certain loops. Then also following the same Tcy and $\pm 1ps$ to get 500,000 Tcy. This changed the equation and how it was solved. The Tcy's are very important to take into account and seeing its effect in timing.

LG